| Name of Student | Roll No. | Date: |
|---|---|---|
| | | |

**Experiment No. 4**

**Tiltle:** To interface keypad with FPGA.

## Pre-lab Requisites:

1. Working of matrix keypad

## Objectives:

1. To write VHDL code 4x4 matrix keypad
2. Simulate with test bench
3. Synthesis, implementation on PLD

## Equipments:

| Sr. No. | Description | Specification |
|---|---|---|
| 1 | System | Processor - 2GHz or Higher, RAM – 2GB or Higher, OS – Windows (32 Bit with SP1 or Higher) |
| 2. | Xilinx | 10.1 |
| 3. | Spartan evaluation board | Spartan 2 PQ208 |
| 4. | SMPS | |

## Theory:

A **keypad** is a set of buttons arranged in a block or "pad" which usually bear digits, symbols and usually a complete set of alphabetical letters. If it mostly contains numbers then it can also be called a numeric keypad. Here we are using **4 X 4 matrix keypad**.

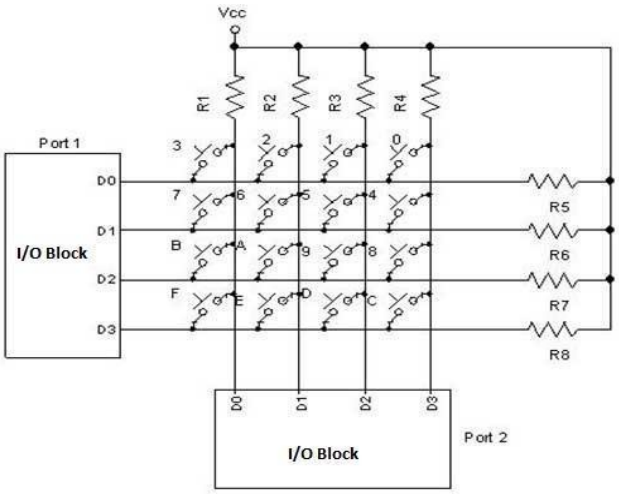



Fig. 4.1 Keypad Interface with FPGA

Fig .1 shows how to interface 4x4 matrix keypad to pins of FPGA. To detect the pressed key, the FPGA grounds all row providing logic 0, and then it reads the columns. If the data read

from columns is D3-D0 = 1111, no key has been pressed and the process continues until a key press has been detected. However, if one of the columns bit has a zero, this means that a key press has been occurred. For example, if D3-D0 = 1101, this means that a key in the D1 column has been pressed. After a key press is detected, the FPGA will go through the process of identifying the key. Starting with the top row, the FPGA grounds it by providing a low to row D0 only; then it reads the columns. If the data read is all 1's, no key in that row is activated and the process is moved to the next row. It grounds the next row, read the columns, and checks for any zero. This process continues until the row is identified. After identification of the row in which the key has been pressed, the next task is to find out which column the pressed key belongs to.

After the key press detection, the FPGA waits for few µs or ns for the bounce and then scans the columns again. This serves two functions:

1. It ensures that the first key press detection was not an erroneous one due to a spike noise.
2. The µs or ns prevents the same key press from being interpreted as a multiple key press.

If after few µs or ns delay the key is still pressed, then it do not go to the next stage to detect another key pressed and shows the same output of the 1st key which is not released. The content of key pressed are then displayed either using seven segment display or LED.

## Procedure:

1. Open Xilinx ISE software and create a new project.
2. Select the appropriate FPGA device and target family.
3. Create a new VHDL source file for keypad interfacing.
4. Write the VHDL code for the 4×4 matrix keypad.
5. Define the input, output, row, and column signals.
6. Implement the clock divider circuit in the VHDL program.
7. Write the scanning logic to detect the pressed key.
8. Assign the corresponding display code for each keypad key.
9. Create a UCF constraint file for pin assignments.
10. Assign the FPGA pins for the clock, reset, rows, columns, and display.
11. Check the VHDL code for syntax errors.
12. Simulate the design using a suitable testbench.
13. Synthesize the design and check the RTL schematic.
14. Download the generated programming file to the Spartan FPGA kit.
15. Press different keypad keys and observe the corresponding output on the seven-segment display.

## VHDL code: -

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity keypad is
    Port (
```

```
            clk     : in STD_LOGIC;
            rst     : in STD_LOGIC;
            row     : inout STD_LOGIC_VECTOR (3 downto 0);
            col     : inout STD_LOGIC_VECTOR (3 downto 0);
            digit   : out STD_LOGIC_VECTOR (3 downto 0);
            display : out STD_LOGIC_VECTOR (7 downto 0)
        );
    end keypad;

    architecture Behavioral of keypad is


  TYPE STATE_TYPE IS (Col1Set, Col2Set, Col3Set, Col4Set);

  SIGNAL coltest : STATE_TYPE;
  SIGNAL data    : STD_LOGIC_VECTOR (7 downto 0);
  SIGNAL rowm    : STD_LOGIC_VECTOR (3 downto 0);
  SIGNAL clock   : STD_LOGIC;

Begin

  Process(clk, rst)
    variable temp : integer range 1 to 10;
  Begin
    if (rst = '0') then
      temp := 1;
    elsif (rising_edge(clk)) then
      temp := temp + 1;

      if (temp = 10) then
        clock <= not clock;
        temp := 1;
      end if;
    end if;
  end process;

  process(clock, rst)
  begin
    if (rst = '0') then
      coltest <= Col1Set;
      rowm    <= "1111";
      data    <= x"00";

    elsif rising_edge(clock) then
      digit <= "1111";

      case coltest is

        when Col1Set =>
          rowm <= "0111";

          case col is
            when "0111" => data <= x"C6"; -- C
            when "1011" => data <= x"A1"; -- D
            when "1101" => data <= x"86"; -- E
```

```
                when "1110" => data <= x"8E"; -- F
                when others  => coltest <= Col2Set;
            end case;

        when Col2Set =>
            rowm <= "1011";

            case col is
                when "0111" => data <= x"C0"; -- 0
                when "1011" => data <= x"F9"; -- 1
                when "1101" => data <= x"A4"; -- 2
                when "1110" => data <= x"B0"; -- 3
                when others  => coltest <= Col3Set;
            end case;

        when Col3Set =>
            rowm <= "1101";

            case col is
                when "0111" => data <= x"99"; -- 4
                when "1011" => data <= x"92"; -- 5
                when "1101" => data <= x"82"; -- 6
                when "1110" => data <= x"F8"; -- 7
                when others  => coltest <= Col4Set;
            end case;

        when Col4Set =>
            rowm <= "1110";

            case col is
                when "0111" => data <= x"80"; -- 8
                when "1011" => data <= x"90"; -- 9
                when "1101" => data <= x"88"; -- A
                when "1110" => data <= x"83"; -- B
                when others  => coltest <= Col1Set;
            end case;

      end case;
    end if;
  end process;

  display <= data;
  row     <= rowm;

end Behavioral;
```

## UCF Constraint File

```
NET "clk" LOC = "P56";
NET "rst" LOC = "P22";

NET "clk" CLOCK_DEDICATED_ROUTE = TRUE;
NET "rst" CLOCK_DEDICATED_ROUTE = FALSE;
```

```
NET "display[0]" LOC = "P48";
NET "display[1]" LOC = "P45";
NET "display[2]" LOC = "P51";
NET "display[3]" LOC = "P47";
NET "display[4]" LOC = "P57";
NET "display[5]" LOC = "P50";
NET "display[6]" LOC = "P59";
NET "display[7]" LOC = "P55";

NET "digit[0]" LOC = "P46";
NET "digit[1]" LOC = "P43";
NET "digit[2]" LOC = "P44";
NET "digit[3]" LOC = "P40";

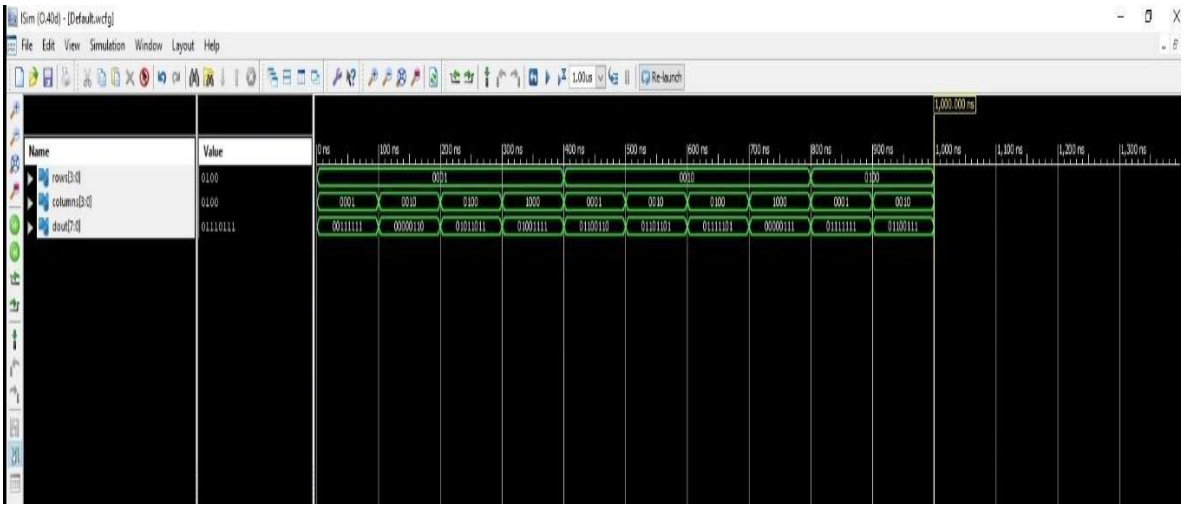
NET "col[0]" LOC = "P58" | PULLUP | DRIVE = 2;
NET "col[1]" LOC = "P62" | PULLUP | DRIVE = 2;
NET "col[2]" LOC = "P61" | PULLUP | DRIVE = 2;
NET "col[3]" LOC = "P66" | PULLUP | DRIVE = 2;

NET "row[0]" LOC = "P64";
NET "row[1]" LOC = "P16";
NET "row[2]" LOC = "P117";
NET "row[3]" LOC = "P75";
```

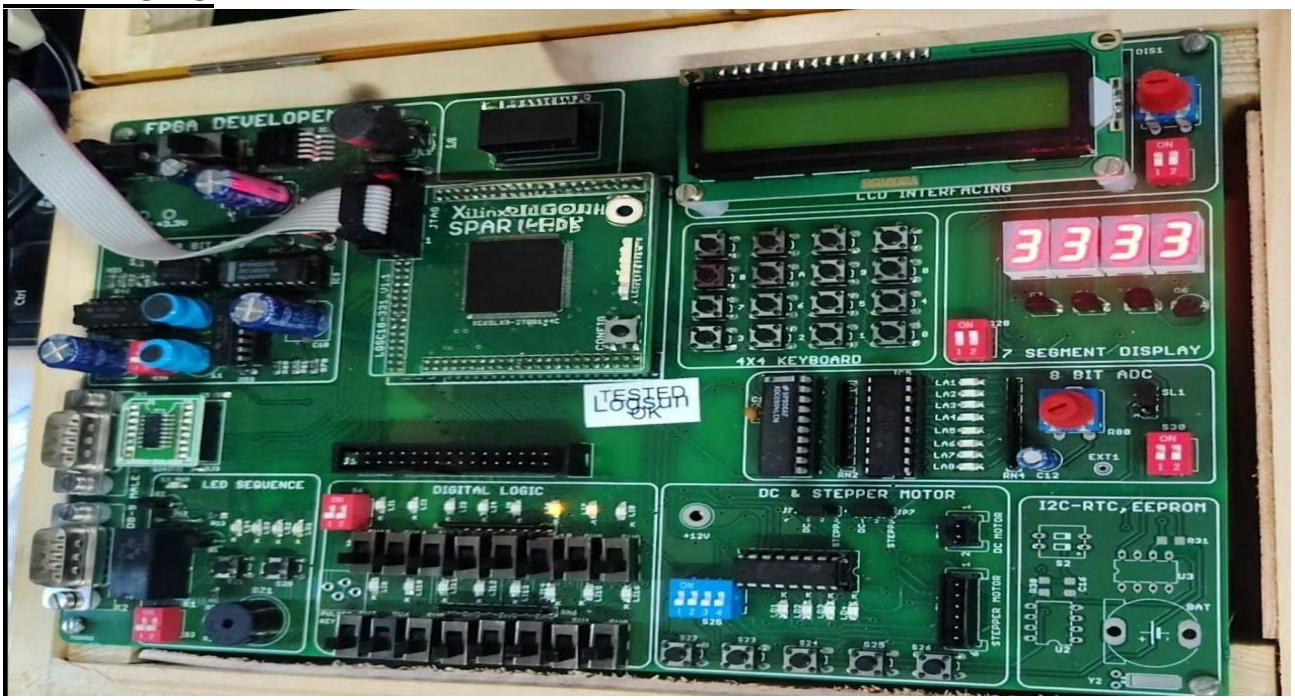
## RTL Schematic: -



## OUTPUT WAVEFORM




## KIT PHOTO



## Conclusion:

**Checked By:**

| Name of Subject Teacher | Sign with Date |
| --- | --- |
| | |